

Challenge Fullstack (React + Java + H2)

Objetivo

Construir una aplicación fullstack que permita gestionar pedidos de clientes.

El sistema debe incluir:

- Backend en Java (Spring Boot)
- Frontend en React (sin Next.js)
- Base de datos H2
- Carga automática de datos iniciales

Nota:

El frontend puede hacerse con herramientas como Vite o Create React App.

Contexto funcional

Una empresa necesita visualizar y gestionar pedidos.

Cada pedido contiene:

- Cliente
 - Fecha
 - Estado (PENDING, PAID, CANCELLED)
 - Total
 - Lista de ítems (nombre, cantidad, precio)
-

Backend (Java + Spring Boot)

Requisitos

1. Modelo de datos

Definir al menos las siguientes entidades:

- Customer
 - id
 - name
 - email
- Order

- id
- customer_id
- date
- status
- total
- OrderItem
 - id
 - order_id
 - product_name
 - quantity
 - price

Relaciones:

- Un customer tiene muchos orders
 - Un order tiene muchos items
-

2. Base de datos

Usar H2.

Se debe cargar información inicial automáticamente al iniciar la aplicación.

Debe haber como mínimo:

- 5 clientes
- 20 pedidos
- 50 items

Se puede usar:

- data.sql
 - CommandLineRunner
 - o alguna herramienta de migración (opcional)
-

3. API REST

Implementar los siguientes endpoints:

- GET /orders
 - Permitir filtros:
 - status

- dateFrom
 - dateTo
 - GET /orders/{id}
 - POST /orders
 - PATCH /orders/{id}/status
-

4. Consideraciones backend

- Usar DTOs para las respuestas
- Manejo básico de errores
- Estructura clara del proyecto

Opcional:

- Paginación (page, size)
 - Logging
-

Frontend (React)

El frontend debe estar hecho en React.

Se puede usar:

- Vite
 - Create React App
 - o configuración propia
-

Requisitos

1. Página principal

Ruta: /orders

Debe:

- Mostrar la lista de pedidos
 - Consumir la API del backend
-

2. Filtros

Permitir filtrar por:

- Estado
- Rango de fechas

Los filtros deben reflejarse en la URL (query params).

3. Detalle de pedido

Ruta: /orders/:id

Debe mostrar:

- Datos del cliente
 - Lista de ítems
 - Total
-

4. Acción sobre pedido

Permitir cambiar el estado de un pedido (por ejemplo, de PENDING a PAID).

Debe:

- Llamar al backend
 - Refrescar la información
-

Opcional (bonus)

- Manejo de loading
 - Optimistic updates
 - Docker (backend + frontend)
 - Validaciones básicas
 - Tests simples
-

Qué se evalúa

- Modelado de datos
- Separación entre frontend y backend
- Consumo de APIs

- Manejo de estado
 - Claridad del código y estructura
-

Qué no es necesario

- Autenticación
 - UI compleja
 - Microservicios
-

Entregable

Repositorio con:

- backend/
- frontend/

Debe incluir un README con:

- Cómo ejecutar el proyecto
- Decisiones técnicas tomadas